
TP7 – VPN avec OpenVPN et PKI

Compte rendu de travaux pratiques

Etudiant : Mazien Adhar

N° etudiant : 20251916

Encadrant : Chattaoui Abdelhakim

Module : TI623/L3 – Mastercamp SR

Date : 24 juin 2026

Table des matières

1	Introduction	2
2	Exercice 1 – Connexion avec la configuration existante	2
3	Exercice 2 – Tunnel sans securite	3
4	Exercice 3 – Creation de la PKI avec Easy-RSA	4
5	Exercice 4 – Cle et certificat pour le serveur	6
6	Exercice 5 – Cle et certificat pour le client	7
7	Exercice 6 – Cle Diffie-Hellman et ta.key	7
8	Exercice 7 – Configuration des fichiers server.conf et client.conf	8
8.1	server.conf	8
8.2	Transfert des fichiers sur le client	9
8.3	client.conf	9
8.4	Test	9
9	Exercice 8 – Table de routage	10
10	Conclusion	11
1	Partie 1 – Manipulations pratiques (8 points)	13
2	Partie 2 – Questions ouvertes (6 points)	19
3	Partie 3 – QCM (6 points)	20

1. Introduction

Ce TP portait sur OpenVPN. L'idée c'était de partir d'une config toute faite pour comprendre comment ca marche, puis de tout reconstruire nous-mêmes avec notre propre PKI via Easy-RSA.

On avait deux VMs sous VMware : Kali en serveur (192.168.200.129) et Ubuntu en client (192.168.200.128).

2. Exercice 1 – Connexion avec la configuration existante

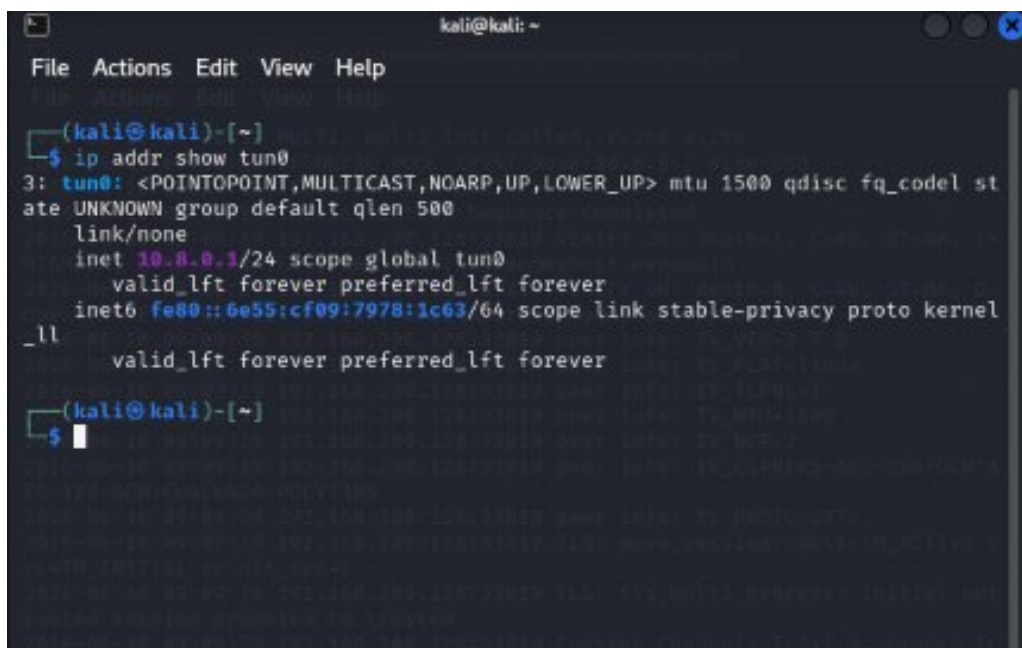
OpenVPN fournit des fichiers d'exemple dans `/usr/share/doc/openvpn/examples/`. La première chose à faire c'était de mettre l'IP du serveur dans `client.conf` :

```
1 sudo nano /usr/share/doc/openvpn/examples/sample-config-files/  
  client.conf  
2 # modifier : remote 192.168.200.129 1194
```

Ensuite on lance le serveur sur Kali et le client sur Ubuntu, tous les deux depuis le dossier `sample-keys` pour que les chemins relatifs des certificats fonctionnent :

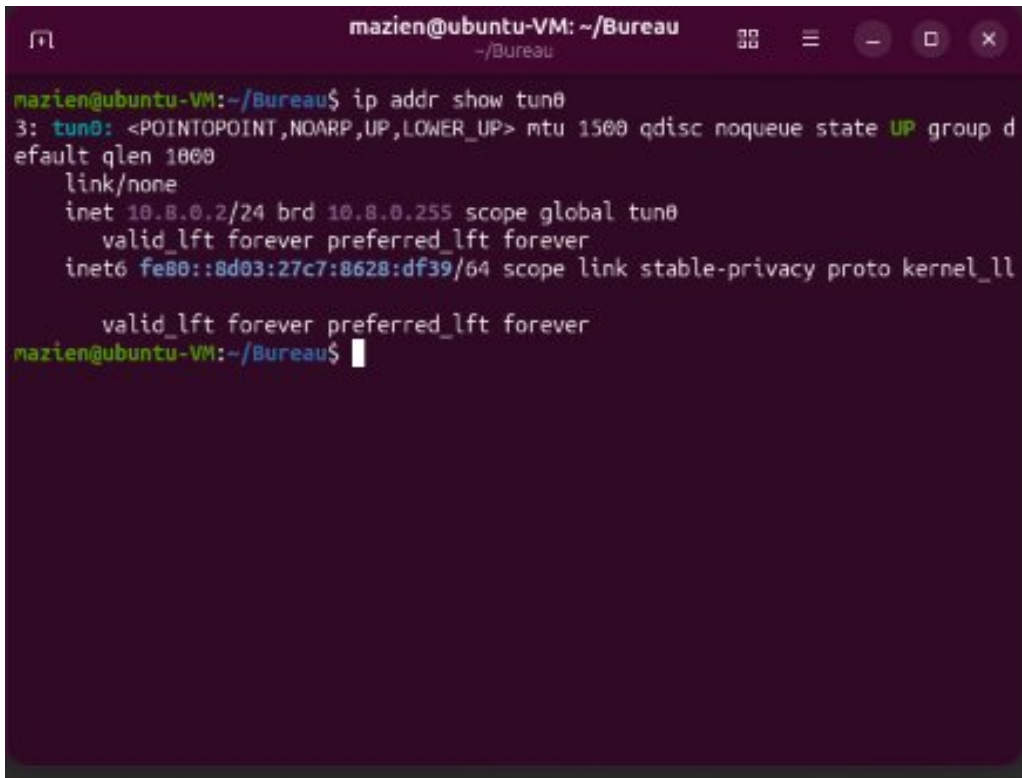
```
1 cd /usr/share/doc/openvpn/examples/sample-keys  
2 sudo openvpn ../sample-config-files/server.conf      # Kali  
3 sudo openvpn ../sample-config-files/client.conf      # Ubuntu
```

Une fois connectés on vérifie les IP sur `tun0` :



```
kali@kali: ~  
File Actions Edit View Help  
File Actions Edit View Help  
(kali@kali)-[~]  
$ ip addr show tun0  
3: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel st  
ate UNKNOWN group default qlen 500  
    link/none  
    inet 10.8.0.1/24 scope global tun0  
        valid_lft forever preferred_lft forever  
    inet6 fe80::6e55:cf09:7978:1c63/64 scope link stable-privacy proto kernel  
_ll  
        valid_lft forever preferred_lft forever  
(kali@kali)-[~]  
$
```

FIGURE 1 – `tun0` cote serveur – 10.8.0.1



```
mazien@ubuntu-VM: ~/Bureau
mazien@ubuntu-VM:~/Bureau$ ip addr show tun0
3: tun0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/none
    inet 10.8.0.2/24 brd 10.8.0.255 scope global tun0
        valid_lft forever preferred_lft forever
    inet6 fe80::8d03:27c7:8628:df39/64 scope link stable-privacy proto kernel_l1
        valid_lft forever preferred_lft forever
mazien@ubuntu-VM:~/Bureau$
```

FIGURE 2 – tun0 cote client – 10.8.0.2

Le tunnel tourne, serveur sur 10.8.0.1 et client sur 10.8.0.2.

3. Exercice 2 – Tunnel sans securite

La on cree un tunnel vraiment basique, juste avec une cle statique partagee, sans aucun certificat. Ca permet de voir ce que ca donne quand il n'y a pas d'authentification.

On genere une cle sur Kali :

```
1 sudo openvpn --genkey secret /etc/openvpn/secret.key
```

On la copie sur Ubuntu, puis on cree les deux configs :

server_nosec.conf :

```
1 dev tun
2 ifconfig 10.8.0.23 10.8.0.89
3 secret /etc/openvpn/secret.key
4 port 1195
5 cipher AES-256-CBC
```

client_nosec.conf :

```
1 dev tun
2 remote 192.168.200.129
3 ifconfig 10.8.0.89 10.8.0.23
```

```

4 secret /etc/openvpn/secret.key
5 port 1195
6 cipher AES-256-CBC
7 allow-deprecated-insecure-static-crypto

1 ping -c 3 10.8.0.23    # test depuis Ubuntu

```



```

mazieng@ubuntu-VM:~/Bureau$ sudo openvpn /etc/openvpn/client_nosec.conf
2026-06-10 15:45:13 DEPRECATED OPTION: The option --secret is deprecated.
2026-06-10 15:45:13 DEPRECATION: No tls-client or tls-server option in configura
tion detected. OpenVPN 2.8 will remove the functionality to run a VPN without TL
S. See the examples section in the manual page for examples of a similar quick s
etup with peer-fingerprint. OpenVPN 2.7 allows using this configuration when usi
ng --allow-deprecated-insecure-static-crypto but you should move to a proper con
figuration using TLS as soon as possible.
2026-06-10 15:45:13 WARNING: file '/etc/openvpn/secret.key' is group or others a
ccessible
2026-06-10 15:45:13 OpenVPN 2.7.0 x86_64-pc-linux-gnu [SSL (OpenSSL)] [LZO] [LZ4
] [EPOLL] [PKCS11] [MH/TKINFO] [AEAD] [DCO]
2026-06-10 15:45:13 library versions: OpenSSL 3.5.5 27 Jan 2026, LZO 2.10
2026-06-10 15:45:13 DCO version: 7.0.0-22-generic #22-Ubuntu SMP PREEMPT_DYNAMIC
Mon May 25 15:54:34 UTC 2026
2026-06-10 15:45:13 TUN/TAP device tun0 opened
2026-06-10 15:45:13 tun/tap device [tun0] opened
2026-06-10 15:45:13 net iface mtu set: mtu 1500 for tun0

```

FIGURE 3 – Logs du tunnel non securise – warnings bien visibles

Ca marche mais OpenVPN affiche des warnings partout : DEPRECATED OPTION: `--secret`, No `tls-client` or `tls-server` option. En gros les donnees circulent sans qu’aucune des deux machines ne puisse verifier l’identite de l’autre – n’importe qui sur le reseau pourrait s’intercaler.

4. Exercice 3 – Creation de la PKI avec Easy-RSA

```

1 sudo apt install easy-rsa
2 sudo cp -r /usr/share/easy-rsa /etc/openvpn/server

```

```
(kali@kali)-[~]
$ tree /etc/openvpn/server
/etc/openvpn/server
├── easy-rsa
│   ├── easyrsa
│   ├── openssl-easyrsa.cnf
│   ├── vars.example
│   └── x509-types
│       ├── ca
│       ├── client
│       ├── code-signing
│       ├── COMMON
│       ├── email
│       ├── kdc
│       ├── server
│       └── serverClient
└── 3 directories, 11 files
```

FIGURE 4 – Arborescence Easy-RSA dans /etc/openvpn/server

```
1 cd /etc/openvpn/server/easy-rsa
2 sudo ./easyrsa init-pki
```

```
(kali@kali)-[/etc/openvpn/server/easy-rsa]
$ sudo tree pki
pki
├── inline
├── issued
├── private
├── reqs
└── vars.example
5 directories, 1 file
```

FIGURE 5 – Structure de pki/ apres init-pki

Ensuite on cree l'autorite de certification. On choisit "autorite" comme Common Name :

```
1 sudo ./easyrsa build-ca
```

```
(kali@kali)-[/etc/openvpn/server/easy-rsa]
$ sudo tree pki
pki
├── ca.crt
├── certs_by_serial
├── index.txt
├── index.txt.attr
├── inline
├── issued
├── private
│   └── ca.key
├── reqs
├── revoked
│   ├── certs_by_serial
│   ├── private_by_serial
│   └── reqs_by_serial
├── serial
└── vars.example
10 directories, 6 files
```

FIGURE 6 – ca.crt et ca.key generes apres build-ca

L'AC cree deux fichiers : ca.crt qu'on va distribuer a tout le monde, et ca.key qui ne doit absolument jamais quitter le serveur.

5. Exercice 4 – Cle et certificat pour le serveur

```
1 sudo ./easyrsa gen-req Efrei nopass
2 sudo ./easyrsa sign-req server Efrei
```

```
(kali@kali)-[/etc/openvpn/server/easy-rsa]
$ sudo tree pki
pki
├── ca.crt
├── certs_by_serial
├── index.txt
├── index.txt.attr
├── inline
├── issued
├── private
│   ├── ca.key
│   └── Efrei.key
├── reqs
│   └── Efrei.req
├── revoked
│   ├── certs_by_serial
│   ├── private_by_serial
│   └── reqs_by_serial
├── serial
└── vars.example
10 directories, 8 files
```

FIGURE 7 – Efrei.key dans private/ et Efrei.req dans reqs/

`gen-req` cree la cle privée du serveur et une demande de signature. `sign-req server` demande a l'AC de signer cette demande – ca produit `pki/issued/Efrei.crt`.

6. Exercice 5 – Cle et certificat pour le client

```
1 sudo ./easyrsa gen-req Client_VPN nopass
2 sudo ./easyrsa sign-req client Client_VPN
```



FIGURE 8 – Client_VPN.crt et Client_VPN.key apres signature

Pareil que pour le serveur, sauf qu'on met `client` au lieu de `server` dans la commande de signature. Ca change le type du certificat dans ses metadonnees X.509.

7. Exercice 6 – Cle Diffie-Hellman et ta.key

```
1 sudo ./easyrsa gen-dh
2 sudo openvpn --genkey secret ta.key
```

```
(kali@kali)-[/etc/openvpn/server/easy-rsa]
$ sudo tree /etc/openvpn/server/easy-rsa
/etc/openvpn/server/easy-rsa
├── easyrsa
├── openssl-easyrsa.cnf
├── pki
│   ├── ca.crt
│   ├── certs_by_serial
│   │   └── A915EF1FF303DF11D75D7CAB09DB8539.pem
│   ├── dh.pem
│   ├── index.txt
│   ├── index.txt.attr
│   ├── index.txt.attr.old
│   ├── index.txt.old
│   ├── inline
│   ├── issued
│   │   └── Client_VPN.crt
│   ├── private
│   │   ├── ca.key
│   │   ├── Client_VPN.key
│   │   └── Efrei.key
│   ├── reqs
│   │   ├── Client_VPN.req
│   │   └── Efrei.req
│   ├── revoked
│   │   ├── certs_by_serial
│   │   ├── private_by_serial
│   │   └── reqs_by_serial
│   ├── serial
│   ├── serial.old
│   └── vars.example
├── ta.key
├── vars.example
└── x509-types
    ├── ca
    ├── client
    ├── code-signing
    ├── COMMON
    ├── email
    ├── kdc
    ├── server
    └── serverClient

12 directories, 28 files

(kali@kali)-[/etc/openvpn/server/easy-rsa]
$
```

FIGURE 9 – dh.pem genere dans pki/ et ta.key a la racine d'easy-rsa

dh.pem contient les parametres pour l'echange de cles pendant la connexion. ta.key c'est une cle supplementaire qui signe les paquets de controle OpenVPN – ca evite que n'importe qui puisse bombarder le serveur de fausses tentatives de connexion.

8. Exercice 7 – Configuration des fichiers server.conf et client.conf

8.1. server.conf

On modifie les chemins pour pointer vers notre PKI. On a mis les chemins absolus pour eviter les problemes selon d'où on lance la commande :

```
1 ca /etc/openvpn/server/easy-rsa/pki/ca.crt
2 cert /etc/openvpn/server/easy-rsa/pki/issued/Efrei.crt
3 key /etc/openvpn/server/easy-rsa/pki/private/Efrei.key
4 dh /etc/openvpn/server/easy-rsa/pki/dh.pem
5 tls-auth /etc/openvpn/server/easy-rsa/ta.key 0
```

8.2. Transfert des fichiers sur le client

Le transfert via SCP a necessite quelques manipulations de droits avant de fonctionner :

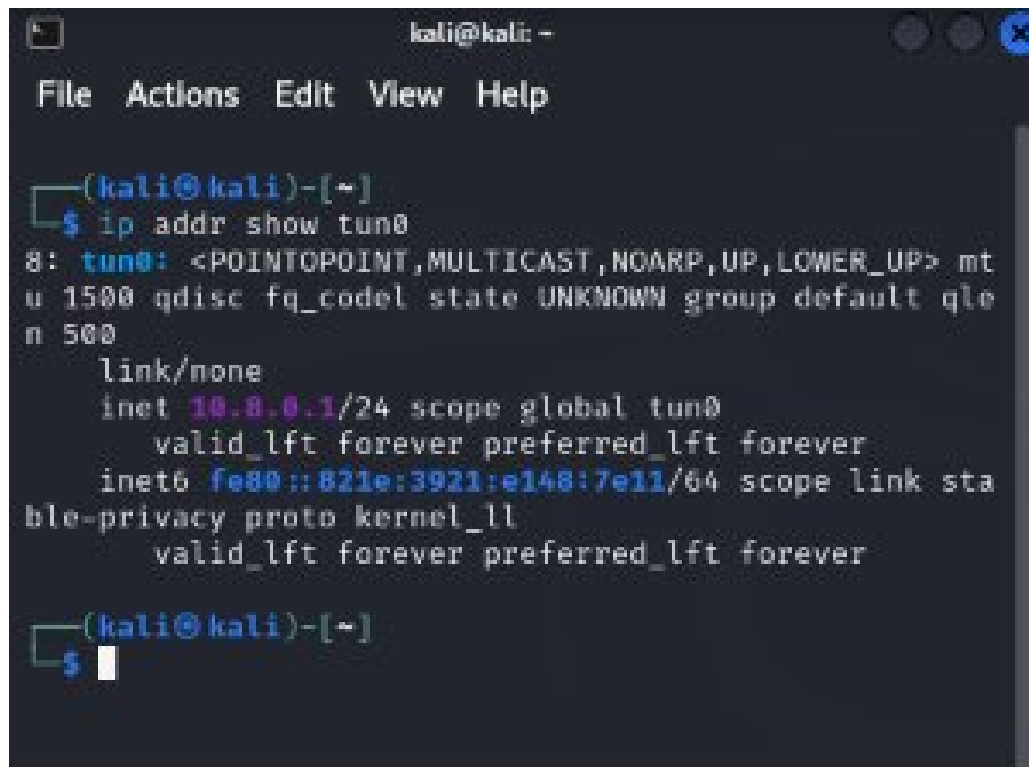
```
1 sudo chmod 644 /tmp/ca.crt /tmp/Client_VPN.crt \
2 /tmp/Client_VPN.key /tmp/ta.key
3 scp /tmp/ca.crt /tmp/Client_VPN.crt \
4 /tmp/Client_VPN.key /tmp/ta.key \
5 mazien@192.168.200.128:/home/mazien/
```

8.3. client.conf

```
1 client
2 dev tun
3 proto udp
4 remote 192.168.200.129 1194
5 resolv-retry infinite
6 nobind
7 ca /etc/openvpn/ca.crt
8 cert /etc/openvpn/Client_VPN.crt
9 key /etc/openvpn/Client_VPN.key
10 tls-auth /etc/openvpn/ta.key 1
11 cipher AES-256-GCM
12 verb 3
```

8.4. Test

```
1 cd /etc/openvpn/server && sudo openvpn server.conf
2 sudo openvpn /etc/openvpn/client.conf
3 ping -c 3 10.8.0.1
```



```

kali@kali: ~
File Actions Edit View Help

(kali@kali)-[~]
$ ip addr show tun0
8: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 500
    link/none
    inet 10.8.0.1/24 scope global tun0
        valid_lft forever preferred_lft forever
    inet6 fe80::821e:3921:e148:7e11/64 scope link stable-privacy proto kernel_ll
        valid_lft forever preferred_lft forever

(kali@kali)-[~]
$

```

FIGURE 10 – tun0 cote serveur : 10.8.0.1



```

mazien@ubuntu-VM: ~/Bureau
~/Bureau
mazien@ubuntu-VM:~/Bureau$ ip addr show tun0
6: tun0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/none
    inet 10.8.0.3/24 brd 10.8.0.255 scope global tun0
        valid_lft forever preferred_lft forever
    inet6 fe80::4381:c474:490e:7c77/64 scope link stable-privacy proto kernel_ll
        valid_lft forever preferred_lft forever
mazien@ubuntu-VM:~/Bureau$

```

FIGURE 11 – tun0 cote client : 10.8.0.3, TLSv1.3 negocié

Cette fois les logs montrent une vraie négociation TLS avec vérification des certificats. Le ping 3/3 confirme que le tunnel fonctionne.

9. Exercice 8 – Table de routage

```
1 route -n
```

```
(kali@kali)-[~]
$ route -n
Kernel IP routing table
Destination      Gateway         Genmask         Flags
Metric Ref      Use Iface
0.0.0.0          192.168.200.2  0.0.0.0         UG
100 0            0 eth0
10.8.0.0         0.0.0.0        255.255.255.0   U
0 0              0 tun0
192.168.200.0    0.0.0.0        255.255.255.0   U
100 0            0 eth0

(kali@kali)-[~]
$
```

FIGURE 12 – route -n sur le serveur

```
mazien@ubuntu-VM:~/Bureau$ route -n
Table de routage IP du noyau
Destination      Passerelle      Genmask         Indic Metric Ref      Use Iface
0.0.0.0          192.168.200.2  0.0.0.0         UG    20100 0            0 ens33
10.8.0.0         0.0.0.0        255.255.255.0   U     0     0            0 tun0
192.168.200.0    0.0.0.0        255.255.255.0   U    100    0            0 ens33
mazien@ubuntu-VM:~/Bureau$
```

FIGURE 13 – route -n sur le client

Après connexion, OpenVPN a ajouté plusieurs routes automatiquement. La route 10.8.0.0 via `tun0` permet d'atteindre les adresses du tunnel. Il y a aussi une route par défaut qui passe par `tun0`, ce qui fait que tout le trafic internet du client passe dans le VPN. La route physique 192.168.200.0 reste sur `ens33` pour que les paquets OpenVPN eux-mêmes puissent circuler – sans ça on aurait une boucle.

10. Conclusion

Le TP m'a permis de voir concrètement pourquoi le tunnel non sécurisé de l'exercice 2 ne vaut rien en pratique. On avait une connexion fonctionnelle mais zéro garantie sur l'identité du serveur. Avec la PKI c'est complètement différent – les deux machines prouvent qui elles sont avant d'échanger quoi que ce soit. La partie qui m'a pris le plus de temps c'était le transfert des fichiers vers le client, entre les problèmes de droits et le SSH qui refusait les connexions par mot de passe.

TP7 – Examen VPN avec OpenVPN et PKI

Feuille de reponses

Etudiant : Mazien Adhar

N° etudiant : 20251916

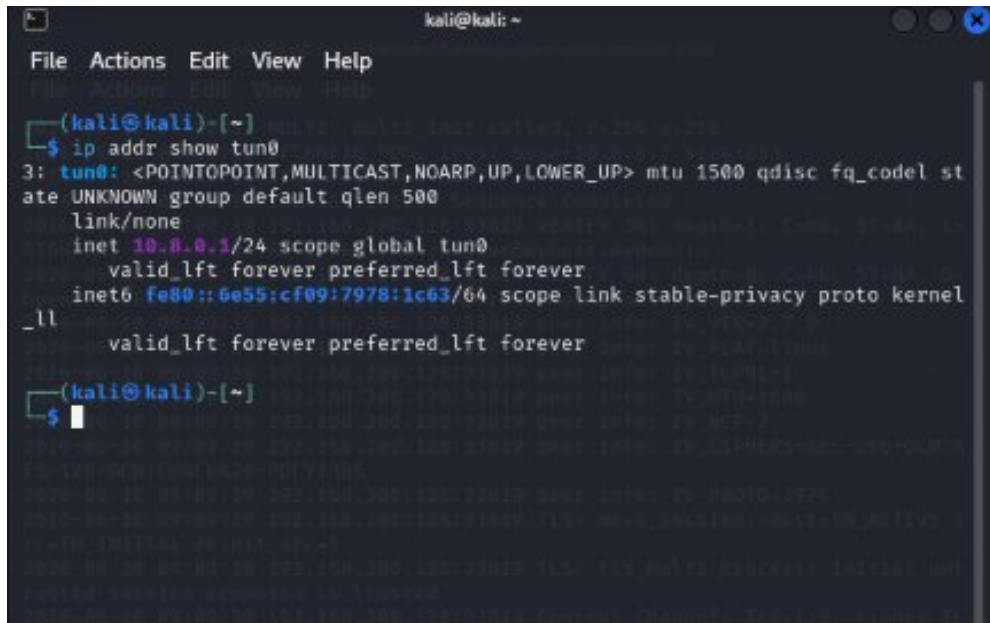
Encadrant : Chattaoui Abdelhakim

Module : TI623/L3 – Mastercamp SR

Date : 24 juin 2026

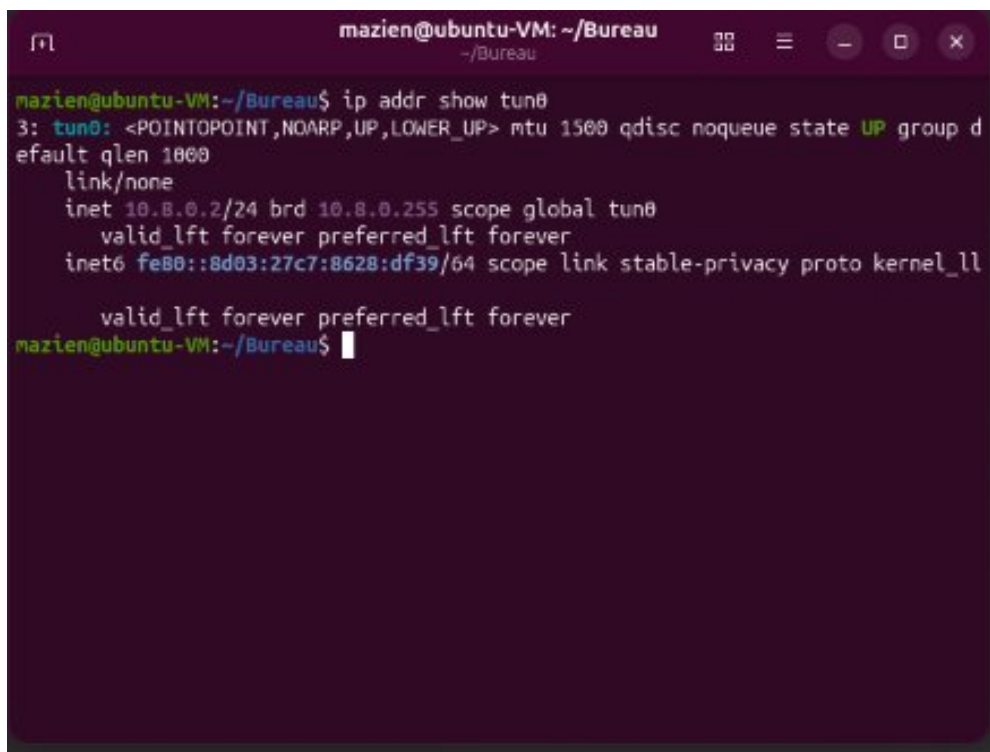
1. Partie 1 – Manipulations pratiques (8 points)

Question 1 (1 pt) – Première connexion VPN, IP attribuées



```
kali@kali: ~  
File Actions Edit View Help  
(kali@kali)-[~]  
$ ip addr show tun0  
3: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel st  
ate UNKNOWN group default qlen 500  
    link/none  
    inet 10.8.0.1/24 scope global tun0  
        valid_lft forever preferred_lft forever  
    inet6 fe80::6e55:cf09:7978:1c63/64 scope link stable-privacy proto kernel  
_ll  
        valid_lft forever preferred_lft forever  
(kali@kali)-[~]  
$
```

FIGURE 1 – tun0 serveur : 10.8.0.1



```
mazien@ubuntu-VM: ~/Bureau  
mazien@ubuntu-VM:~/Bureau$ ip addr show tun0  
3: tun0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group d  
efault qlen 1000  
    link/none  
    inet 10.8.0.2/24 brd 10.8.0.255 scope global tun0  
        valid_lft forever preferred_lft forever  
    inet6 fe80::8d03:27c7:8628:df39/64 scope link stable-privacy proto kernel_ll  
        valid_lft forever preferred_lft forever  
mazien@ubuntu-VM:~/Bureau$
```

FIGURE 2 – tun0 client : 10.8.0.2

Question 2 (1 pt) – Configuration non securisee, test connexion

```
mazien@ubuntu-VM:~/Bureau$ sudo openvpn /etc/openvpn/client/nosec.conf
2026-06-10 15:45:13 DEPRECATED OPTION: The option --secret is deprecated.
2026-06-10 15:45:13 DEPRECATION: No tls-client or tls-server option in configura
tion detected. OpenVPN 2.8 will remove the functionality to run a VPN without TL
S. See the examples section in the manual page for examples of a similar quick s
etup with peer-fingerprint. OpenVPN 2.7 allows using this configuration when usi
ng --allow-deprecated-insecure-static-crypto but you should move to a proper con
figuration using TLS as soon as possible.
2026-06-10 15:45:13 WARNING: file '/etc/openvpn/secret.key' is group or others a
ccessible
2026-06-10 15:45:13 OpenVPN 2.7.0 x86_64-pc-linux-gnu [SSL (OpenSSL)] [LZO] [LZ4
] [EPOLL] [PKCS11] [MH/PKTINFO] [AEAD] [DCO]
2026-06-10 15:45:13 library versions: OpenSSL 3.5.5 27 Jan 2026, LZO 2.10
2026-06-10 15:45:13 DCO version: 7.0.0-22-generic #22-Ubuntu SMP PREEMPT_DYNAMIC
Mon May 25 15:54:34 UTC 2026
2026-06-10 15:45:13 TUN/TAP device tun0 opened
2026-06-10 15:45:13 tun/tap device [tun0] opened
2026-06-10 15:45:13 net iface mtu set: mtu 1500 for tun0
```

FIGURE 3 – Tunnel non securise actif, warnings OpenVPN visibles

Question 3 (1 pt) – PKI Easy-RSA, structure des fichiers

```
(kali@kali)-[~]
$ tree /etc/openvpn/server
/etc/openvpn/server
├── easy-rsa
│   ├── easyrsa
│   ├── openssl-easyrsa.cnf
│   ├── vars.example
│   └── x509-types
│       ├── ca
│       ├── client
│       ├── code-signing
│       ├── COMMON
│       ├── email
│       ├── kdc
│       ├── server
│       └── serverClient
└── 3 directories, 11 files
```

FIGURE 4 – Arborescence Easy-RSA apres installation

```
(kali@kali)-[/etc/openvpn/server/easy-rsa]
$ sudo tree pki
pki
├── inline
├── issued
├── private
├── reqs
└── vars.example
5 directories, 1 file
```

FIGURE 5 – pki/ apres init-pki

Question 4 (1 pt) – Cle et certificat serveur, signature AC

```
(kali@kali)-[/etc/openvpn/server/easy-rsa]
$ sudo tree pki
pki
├── ca.crt
├── certs_by_serial
├── index.txt
├── index.txt.attr
├── inline
├── issued
├── private
│   └── ca.key
├── reqs
├── revoked
│   ├── certs_by_serial
│   ├── private_by_serial
│   └── reqs_by_serial
├── serial
└── vars.example
10 directories, 6 files
```

FIGURE 6 – ca.crt et ca.key apres build-ca

```
(kali@kali)-[/etc/openvpn/server/easy-rsa]
$ sudo tree pki
pki
├── ca.crt
├── certs_by_serial
├── index.txt
├── index.txt.attr
├── inline
├── issued
├── private
│   ├── ca.key
│   └── Efrei.key
├── reqs
│   └── Efrei.req
├── revoked
│   ├── certs_by_serial
│   ├── private_by_serial
│   └── reqs_by_serial
├── serial
└── vars.example
10 directories, 8 files
```

FIGURE 7 – Efrei.key et Efrei.req generes

Question 5 (1 pt) – Cle et certificat client, signature AC

```
(kali@kali)-[/etc/openvpn/server/easy-rsa]
$ sudo tree pki
pki
├── ca.crt
├── certs_by_serial
│   └── A915EF1FF303DF11D75D7CAB09DBB539.pem
├── index.txt
├── index.txt.attr
├── index.txt.attr.old
├── index.txt.old
├── inline
├── issued
│   └── Client_VPN.crt
├── private
│   ├── ca.key
│   ├── Client_VPN.key
│   └── Efrei.key
├── reqs
│   ├── Client_VPN.req
│   └── Efrei.req
├── revoked
│   ├── certs_by_serial
│   ├── private_by_serial
│   └── reqs_by_serial
├── serial
├── serial.old
└── vars.example

10 directories, 15 files
```

FIGURE 8 – Client_VPN.crt et Client_VPN.key signes

Question 6 (1 pt) – Cle DH et ta.key, emplacement

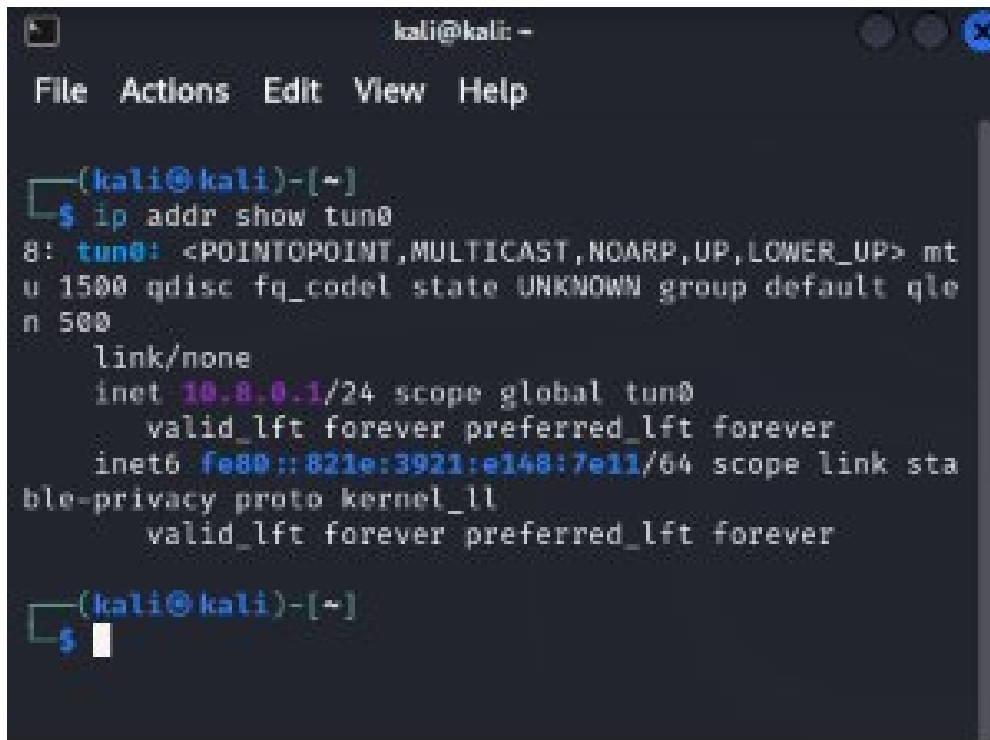
```
(kali@kali)-[/etc/openvpn/server/easy-rsa]
$ sudo tree /etc/openvpn/server/easy-rsa
/etc/openvpn/server/easy-rsa
├── easyrsa
├── openssl-easyrsa.cnf
├── pki
│   ├── ca.crt
│   ├── certs_by_serial
│   │   └── A915EF1FF303DF11D75D7CAB09DB8539.pem
│   ├── dh.pem
│   ├── index.txt
│   ├── index.txt.attr
│   ├── index.txt.attr.old
│   ├── index.txt.old
│   ├── inline
│   ├── issued
│   │   └── Client_VPN.crt
│   ├── private
│   │   ├── ca.key
│   │   ├── Client_VPN.key
│   │   └── Efrei.key
│   ├── reqs
│   │   ├── Client_VPN.req
│   │   └── Efrei.req
│   ├── revoked
│   │   ├── certs_by_serial
│   │   ├── private_by_serial
│   │   └── reqs_by_serial
│   ├── serial
│   ├── serial.old
│   └── vars.example
├── ta.key
├── vars.example
└── x509-types
    ├── ca
    ├── client
    ├── code-signing
    ├── COMMON
    ├── email
    ├── kdc
    ├── server
    └── serverClient

12 directories, 28 files

(kali@kali)-[/etc/openvpn/server/easy-rsa]
$
```

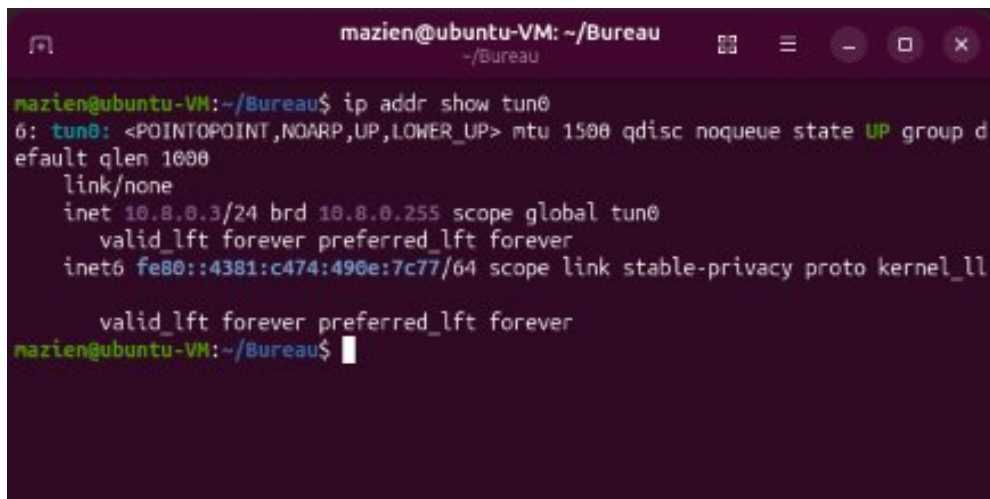
FIGURE 9 – dh.pem dans pki/ et ta.key a la racine d'easy-rsa

Question 7 (1 pt) – server.conf et client.conf, test tunnel



```
kali@kali: -  
File Actions Edit View Help  
  
(kali@kali)-[~]  
$ ip addr show tun0  
8: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mt  
u 1500 qdisc fq_codel state UNKNOWN group default qlen  
500  
    link/none  
    inet 10.8.0.1/24 scope global tun0  
        valid_lft forever preferred_lft forever  
    inet6 fe80::821e:3921:e148:7e11/64 scope link sta  
ble-privacy proto kernel_ll  
        valid_lft forever preferred_lft forever  
  
(kali@kali)-[~]  
$
```

FIGURE 10 – tun0 serveur : 10.8.0.1



```
mazien@ubuntu-VM: ~/Bureau  
- /Bureau  
  
mazien@ubuntu-VM:~/Bureau$ ip addr show tun0  
6: tun0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group d  
efault qlen 1000  
    link/none  
    inet 10.8.0.3/24 brd 10.8.0.255 scope global tun0  
        valid_lft forever preferred_lft forever  
    inet6 fe80::4381:c474:490e:7c77/64 scope link stable-privacy proto kernel_ll  
        valid_lft forever preferred_lft forever  
mazien@ubuntu-VM:~/Bureau$
```

FIGURE 11 – tun0 client : 10.8.0.3

Question 8 (1 pt) – Table de routage, impact sur le reseau

```
(kali@kali)-[~]
$ route -n
Kernel IP routing table
Destination      Gateway         Genmask         Flags
Metric Ref      Use Iface
0.0.0.0          192.168.200.2  0.0.0.0         UG
100 0            0 eth0
10.8.0.0          0.0.0.0        255.255.255.0   U
0 0              0 tun0
192.168.200.0    0.0.0.0        255.255.255.0   U
100 0            0 eth0
```

FIGURE 12 – route -n serveur

```
mazien@ubuntu-VM:~/Bureau$ route -n
Table de routage IP du noyau
Destination      Passerelle      Genmask         Indic Metric Ref      Use Iface
0.0.0.0          192.168.200.2  0.0.0.0         UG    20100 0            0 ens33
10.8.0.0          0.0.0.0        255.255.255.0   U      0 0            0 tun0
192.168.200.0    0.0.0.0        255.255.255.0   U     100 0            0 ens33
```

FIGURE 13 – route -n client

2. Partie 2 – Questions ouvertes (6 points)

Question 1 (2 pts) – Role de la PKI dans la securisation d'un tunnel VPN

La PKI sert à ce que les deux machines puissent vérifier qu'elles parlent bien à qui elles croient. Dans l'exercice 2 on n'avait pas ça – le tunnel marchait mais rien n'empêche un attaquant sur le même réseau de se faire passer pour le serveur, le client n'aurait aucun moyen de s'en rendre compte.

Avec la PKI chaque machine a un certificat signé par notre AC. Quand la connexion s'établit, OpenVPN vérifie que le certificat en face a bien été signé par cette même AC. Si ce n'est pas le cas la connexion est coupée directement.

Ça couvre aussi d'autres cas. Si une machine est compromise et que sa clé privée est volée, on peut révoquer son certificat – elle ne pourra plus se connecter au tunnel même avec les anciens fichiers. Ça évite d'avoir à tout reconfigurer.

Sans PKI on a juste un tuyau chiffré, mais on ne sait pas à qui on parle vraiment.

Avec, on a a la fois le chiffrement et l'authentification.

Question 2 (2 pts) – Utilite de la cle Diffie-Hellman dans OpenVPN

Le fichier `dh.pem` sert a l'echange de cles au moment de la connexion. Le principe c'est que les deux machines calculent une cle de session commune sans jamais l'envoyer en clair sur le reseau – ce qui est un peu contra-intuitif mais ca fonctionne vraiment.

Chaque cote genere un nombre secret, en deduit une valeur publique, et l'envoie a l'autre. Avec cette valeur publique et son propre secret, chaque machine arrive au meme resultat : la cle de session. Quelqu'un qui ecouterait tous les echanges ne pourrait pas recalculer cette cle.

L'avantage concret c'est que meme si quelqu'un vole la cle privee du serveur apres coup, ca ne lui servira pas a grand chose pour dechiffrer les sessions passees. Les cles de session ne sont pas stockees et une nouvelle est generee a chaque connexion.

Dans notre TP le `dh.pem` 2048 bits est utilise pendant le handshake pour que les deux cotes tombent d'accord sur les cles AES-256-GCM sans les avoir jamais echangees directement.

Question 3 (2 pts) – Analyse de la table de routage apres etablisement du tunnel

Apres `route -n` on voit plusieurs nouvelles entrees ajoutes par OpenVPN. La principale c'est `10.8.0.0` via `tun0` – tout ce qui va vers le reseau du tunnel passe par l'interface chiffree.

Il y a aussi une route par default via `tun0`, donc tout le trafic internet du client passe dans le VPN. Et la route `192.168.200.0` reste sur `ens33` – c'est oblige, sinon les paquets OpenVPN eux-memes n'auraient pas de chemin pour atteindre le serveur et on se retrouverait avec une boucle infinie.

En gros le client voit le reseau `10.8.0.x`, mais dessous tout passe en UDP sur le `192.168.200.x`.

3. Partie 3 – QCM (6 points)

Q1. Quel outil est utilise pour generer une PKI dans ce TP ?

☐ OpenSSL ☒ Easy-RSA ☐ GPG ☐ Wireshark

Q2. Quel fichier contient la configuration du client OpenVPN ?

☐ `/etc/openvpn/server.conf` ☒ `/etc/openvpn/client.conf` ☐ `/etc/hosts`
☐ `/etc/resolv.conf`

Q3. Quel est le role de la cle `ta.key` ?

☐ Chiffrer les donnees ☒ **Authentifier les connexions TLS** ☐ Generer des certificats ☐ Definir les routes

Q4. Pourquoi signer les certificats avec une AC ?

☐ Pour les rendre lisibles par le systeme ☒ **Pour garantir leur authenticite**
☐ Pour les convertir en .pem ☐ Pour les rendre compatibles HTTP

Q5. Que permet la commande route -n ?

☐ Generer une cle ☐ Lister les certificats ☒ **Afficher la table de routage**
☐ Demarrer le service VPN

Q6. Quel est l'interet d'un VPN ?

☐ Accelerer la connexion Internet ☒ **Securiser les echanges entre deux machines distantes** ☐ Remplacer un pare-feu ☐ Supprimer les mots de passe